Introduction to Operations Research, Determinisitc Models

Lecture 6: Dual Simplex Method and Large-scale linear programming

Department of Industrial Engineering, Tsinghua University

Instructor: Junyi Liu

Oct 22nd, 2024

Recap: dual problem

In the last lecture, we introduce

Primal problem (general form)

$$\begin{array}{ll} \textbf{maximize} & Z = \mathbf{c}^\top \mathbf{x} \\ \text{subject to} & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x}(\in \mathbb{R}^n) \geq 0 \end{array}$$

Dual problem

$$\begin{array}{ll} \textbf{minimize} & W = \mathbf{b}^\top \mathbf{y} \\ \text{subject to} & \mathbf{A}^\top \mathbf{y} \geq \mathbf{c} \\ & \mathbf{y}(\in \mathbb{R}^m) \geq 0 \end{array}$$

For the notational convenience, let us rewrite $\mathbf{c}$ and $\mathbf{y}$ as row vector, and maintain $\mathbf{b}$ and $\mathbf{x}$ as column vectors. We rewrite the primal and dual problems as follows.

Primal problem (general form)

$$\begin{array}{ll} \textbf{maximize} & Z = \mathbf{c}\,\mathbf{x} \\ \text{subject to} & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x}(\in \mathbb{R}^n) \geq 0 \end{array}$$

Dual problem

$$\begin{array}{ll} \textbf{minimize} & W = \mathbf{y}\,\mathbf{b} \\ \text{subject to} & \mathbf{y}\mathbf{A} \geq \mathbf{c} \\ & \mathbf{y}(\in \mathbb{R}^m) \geq 0 \end{array}$$

Recap: weak and strong duality properties

Weak duality

If $\mathbf{x}$ is a feasible solution for the primal problem and $\mathbf{y}$ is a feasible solution for the dual problem, then $\mathbf{c}\,\mathbf{x} \leq \mathbf{y}\,\mathbf{b}$

Corollary 1

Let $\bar{\mathbf{x}}$ be a feasible solution for the primal problem and $\bar{\mathbf{y}}$ be a feasible solution for the dual problem. If $\mathbf{c}\,\bar{\mathbf{x}} = \bar{\mathbf{y}}\,\mathbf{b}$, then $\bar{\mathbf{x}}$ is optimal for the primal and $\bar{\mathbf{y}}$ is optimal for the dual.

Corollary 2

If one problem has feasible solutions and an unbounded objective function (and so no optimal solution), then the other problem has no feasible solutions.

Recap: weak and strong duality properties

Weak duality

If $\mathbf{x}$ is a feasible solution for the primal problem and $\mathbf{y}$ is a feasible solution for the dual problem, then $\mathbf{c}\,\mathbf{x} \leq \mathbf{y}\,\mathbf{b}$

Corollary 1

Let $\bar{\mathbf{x}}$ be a feasible solution for the primal problem and $\bar{\mathbf{y}}$ be a feasible solution for the dual problem. If $\mathbf{c}\,\bar{\mathbf{x}} = \bar{\mathbf{y}}\,\mathbf{b}$, then $\bar{\mathbf{x}}$ is optimal for the primal and $\bar{\mathbf{y}}$ is optimal for the dual.

Corollary 2

If one problem has feasible solutions and an unbounded objective function (and so no optimal solution), then the other problem has no feasible solutions.

Strong duality

If $\mathbf{x}^*$ is an optimal solution for the primal problem of LP and $\mathbf{y}^*$ is an optimal solution for its dual problem, then $\mathbf{c}\,\mathbf{x}^* = \mathbf{y}^*\,\mathbf{b}$.

Recap: duality theorem

Dual of the dual problem is the primal problem.
The relationships of the dual problem to the primal problem also hold in reverse

Duality theorem

The following are the only possible relationships between the primal and dual problems.

1. If one problem has feasible solutions and a bounded objective function (and so has an optimal solution), then so does the other problem. Moreover, both the weak and strong duality properties hold.
2. If one problem has feasible solutions and an unbounded objective function (and so no optimal solution), then the other problem has no feasible solutions.
3. If one problem has no feasible solutions, then the other problem has either no feasible solutions or an unbounded objective function.

Recap: complementary solution property

	Basic variable	row	Coefficient			RHS
			Z	$\mathbf{x}$	$\mathbf{x}_S$	
Some	Z	(0)	1	$\mathbf{y}^*\mathbf{A} - \mathbf{c}$	$\mathbf{y}^*$	$\mathbf{y}^*\mathbf{b}$
	$\mathbf{x}_B$		0	$\mathbf{B}^{-1}\mathbf{A}$	$\mathbf{B}^{-1}$	$\mathbf{B}^{-1}\mathbf{b}$

Observation: For each tableau, the coefficients $\mathbf{y}^*$ of the slack variables is a CP solution for the dual problem, because m out of $m+n$ coefficients in row 0 equal 0, and thus $\mathbf{y}^*$ lies at the intersection of m constraint boundaries out of $m+n$ constraints ($\mathbf{y}\,\mathbf{A} \geq \mathbf{c}$, $\mathbf{y} \geq 0$).

Complementary solutions property: At each iteration, the simplex method simultaneously identifies a CPF solution $\mathbf{x}$ for the primal problem and a complementary CP solution $\mathbf{y}^*$ for the dual problem, where $\mathbf{c}\,\mathbf{x} = \mathbf{y}^*\,\mathbf{b}$. If $\mathbf{x}$ is not optimal for the primal problem, then $\mathbf{y}^*$ is not feasible for the dual problem.

Complementary optimal solutions property: At the final iteration, the simplex method simultaneously identifies an optimal solution $\mathbf{x}^*$ for the primal problem and a complementary optimal solution $\mathbf{y}^*$ for the dual problem (found in row 0, the coefficients of the slack variables), where $\mathbf{c}\,\mathbf{x}^* = \mathbf{y}^*\,\mathbf{b}$.

Recap: complementary basic solution property

Dual problem

minimize	$W = \mathbf{y}\,\mathbf{b}$
subject to	$\mathbf{y}\mathbf{A} \geq \mathbf{c}$
	$\mathbf{y}(\in \mathbb{R}^m) \geq 0$

Augmented dual problem

minimize	$W = \mathbf{y}\,\mathbf{b}$
subject to	$\mathbf{y}\mathbf{A} - \mathbf{y}_e = \mathbf{c}$
	$\mathbf{y}(\in \mathbb{R}^m) \geq 0, \mathbf{y}_e \geq 0.$

Recall that a basic solution is an augmented corner-point solution.

Observation: For each tableau, the coefficients $(\mathbf{y}^*, \mathbf{y}_e^*) \triangleq (\mathbf{y}^*, \mathbf{y}^*\mathbf{A} - \mathbf{c})$ in row 0 is a basic solution for the dual problem, because $\mathbf{y}^*$ is a CP solution, and $\mathbf{y}^*\mathbf{A} - \mathbf{y}_e^* = \mathbf{c}$.

	Basic variable	row	Coefficient			RHS
			Z	$\mathbf{x}$	$\mathbf{x}_S$	
Some	Z	(0)	1	$\mathbf{y}^*\mathbf{A} - \mathbf{c}$	$\mathbf{y}^*$	$\mathbf{y}^*\mathbf{b}$
	$\mathbf{x}_B$		0	$\mathbf{B}^{-1}\mathbf{A}$	$\mathbf{B}^{-1}$	$\mathbf{B}^{-1}\mathbf{b}$

Recap: complementary basic solution property

Complementary basic solutions property:

• Each basic solution (feasible or not) in the primal problem has a complementary basic solution (feasible or not) in the dual problem, with the same objective function values. Given row 0 of the simplex tableau for the primal basic solution, the complementary dual basic solution is $\boxed{(\mathbf{y}^*, \mathbf{y}^*\mathbf{A} - \mathbf{c})}$.

• If the basic solution is not optimal for the primal problem, then the complementary dual basic solution is not feasible for the dual problem.

Complementary optimal basic solutions property: An optimal basic solution in the primal problem has a complementary optimal basic solution in the dual problem, with the same objective function values.

In this lecture, you'll learn

- Dual simplex method
- Column generation method
- Constraint generation method

Dual simplex method

Dual simplex method

The simplex method:

- starts with a primal feasible solution which corresponds to a dual infeasiblesolution;
- it maintains primal feasibility during the process;
- obtains the optimality when dual feasibility **(row 0 nonnegative)** is attained.

Dual simplex method

The simplex method:

- starts with a primal feasible solution which corresponds to a dual infeasiblesolution;
- it maintains primal feasibility during the process;
- obtains the optimality when dual feasibility **(row 0 nonnegative)** is attained.

Dual simplex method:

- starts with a dual feasible solution corresponding to a primal infeasible solution;
- it maintains dual feasibility during the process;
- obtains the optimality when primal feasibility **(RHS is nonnegative)** is attained.

Dual simplex method can be regarded as a simplex method applied to the complementary basic solutions in the dual problem.

Dual simplex method

Advantages:

• It may be easier to obtain the initial dual BF solution than the initial primal BF solution. Particular, when artificial variables are not present, it may require a fewer number of iterations.

Dual simplex method

Advantages:

• It may be easier to obtain the initial dual BF solution than the initial primal BF solution. Particular, when artificial variables are not present, it may require a fewer number of iterations.

• In sensitivity analysis, when RHS is changed or a new constraint is added to a solved LP, the current primal basic solution may not be feasible, but the current dual basic solution is feasible (row 0 coefficients are nonnegative). Then the dual simplex method is applicable starting with the dual BF solution, which is usually much more quicker than the simplex method from scratch.

Dual simplex method

Advantages:

• It may be easier to obtain the initial dual BF solution than the initial primal BF solution. Particular, when artificial variables are not present, it may require a fewer number of iterations.

• In sensitivity analysis, when RHS is changed or a new constraint is added to a solved LP, the current primal basic solution may not be feasible, but the current dual basic solution is feasible (row 0 coefficients are nonnegative). Then the dual simplex method is applicable starting with the dual BF solution, which is usually much more quicker than the simplex method from scratch.

• The computational efficiency of the simplex method is largely affected by the number of constraints (m) rather than the number of decision variables (n), as every iteration involves the update of $\mathbf{B}^{-1}_{m\times m}$. For a certain huge LP problem ($m \gg n$), the dual simplex method is much more efficient than the simplex method.

Dual simplex method

Consider the dual of Wyndor problem

$$\begin{aligned} \text{minimize} \quad & W = 4y_1 + 12y_2 + 18y_3 \\ \text{subject to} \quad & \begin{pmatrix} 1 & 0 & 3 \\ 0 & 2 & 2 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} \geq \begin{pmatrix} 3 \\ 5 \end{pmatrix} \\ & \mathbf{y} \geq \mathbf{0} \end{aligned}$$

Since negative RHS is allowed in the dual simplex method, we simply convert the above problem into a maximization problem with $\leq$ functional constraints.

$$\begin{aligned} \text{maximize} \quad & Z = -4y_1 - 12y_2 - 18y_3 \\ \text{subject to} \quad & \begin{pmatrix} -1 & 0 & -3 \\ 0 & -2 & -2 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} \leq \begin{pmatrix} -3 \\ -5 \end{pmatrix} \\ & \mathbf{y} \geq \mathbf{0} \end{aligned}$$

We start with an initial basic solution $(0, 0, 0, -3, -5)$. (the coefficients in row (0) are zero for basic variables and nonnegative for nonbasic variables).

Dual simplex method

Iteration	Basic variable	row	Coefficient						RHS
			Z	y_1	y_2	y_3	y_4	y_5	
0	Z	(0)	1	4	12	18	0	0	0
	y_4	(1)	0	-1	0	-3	1	0	-3
	y_5	(2)	0	0	-2	-2	0	1	-5
1	Z	(0)	1	4	0	6	0	6	-30
	y_4	(1)	0	-1	0	-3	1	0	-3
	y_2	(2)	0	0	1	1	0	-1/2	5/2

Step 1. Determine the leaving basic variable: select the basic variable that has the most negative value. ($-5 < -3$, so y_5 is the leaving basic variable.)
Step 2. Determine the entering basic variable: select the nonbasic variable whose coefficient in row(0) reaches zero first when an increasing multiple of the equation containing the leaving basic variable is added to Eq. (0). ($12/2 < 18/2$, so y_2 is the entering basic variable)
Step 3. Determine the new basic solution.

Column Generation Method

Large-scale linear programming

- Linear programs occurring in practice can be extremely large, though the vast majority of the matrix is superfluous.
- To solve a particular instance, we only need
  - constraints that are binding at optimality;
  - variables that are basic at optimality.

Large-scale linear programming

- Linear programs occurring in practice can be extremely large, though the vast majority of the matrix is superfluous.
- To solve a particular instance, we only need
  - constraints that are binding at optimality;
  - variables that are basic at optimality.
- If we only had the knowledge of these variables and constraints from the start, we could solve very easily.
- Unfortunately, we don't know which variables and constraints these are.

Column generation method

$$\begin{array}{ll} \textbf{maximize} & Z = \mathbf{c}\,\mathbf{x} \\ \text{subject to} & \displaystyle\sum_{j=1}^{n} \mathbf{A}_j \mathbf{x}_j \leq \mathbf{b} \\ & \mathbf{x}(\in \mathbb{R}^n) \geq 0 \end{array}$$

- For problems with **large numbers of variables**, there are two main difficulties.
  - the time required just to generate the coefficient matrix.
  - the time required to calculate row 0 coefficient at each iteration.

Column generation method

$$\begin{array}{ll} \textbf{maximize} & Z = \mathbf{c}\,\mathbf{x} \\ \text{subject to} & \displaystyle\sum_{j=1}^{n} \mathbf{A}_j \mathbf{x}_j \leq \mathbf{b} \\ & \mathbf{x}(\in \mathbb{R}^n) \geq 0 \end{array}$$

- For problems with **large numbers of variables**, there are two main difficulties.
  - the time required just to generate the coefficient matrix.
  - the time required to calculate row 0 coefficient at each iteration.

- Idea of **column generation method**:
  - start with a subset of "promising" columns from n columns.

At each iteration,
  - solve the LP with just these columns.
  - from the remaining columns, add a column with the largest positive rate of improvement $\mathbf{c}_B \mathbf{B}^{-1} \mathbf{A}_j - \mathbf{c}_j$.

Column generation method

The **core** of delayed column generation is based on the ability to generate a column with the largest positive rate of improvement.

Column generation method

The **core** of delayed column generation is based on the ability to generate a column with the largest positive rate of improvement.

- **Seeking the largest positive rate of improvement** is an optimization problem.
- If we can solve this optimization problem efficiently (e.g., MIP/dynamic programming), then we can solve the LP without explicitly listing the columns.

Column generation method

The **core** of delayed column generation is based on the ability to generate a column with the largest positive rate of improvement.

• **Seeking the largest positive rate of improvement** is an optimization problem.

• If we can solve this optimization problem efficiently (e.g., MIP/dynamic programming), then we can solve the LP without explicitly listing the columns.

• There are many variants of this basic algorithm. All are based in the ability to generate a column with largest positive rate of improvement, given the current dual prices.

Column generation algorithm

Start with a subset of columns indexed by the set $\mathcal{I} \subseteq [n]$. At each iteration $r = 1, \ldots,$

- Consider the restricted problem obtained by considering only the subset of the columns indexed by set I

$$
\begin{aligned}
\textbf{maximize} \quad & \sum_{j \in I} \mathbf{c}_j \, \mathbf{x}_j \\
\text{subject to} \quad & \sum_{j \in I} \mathbf{A}_j \mathbf{x}_j \leq \mathbf{b} \\
& \mathbf{x} (\in \mathbb{R}^n) \geq 0
\end{aligned}
$$

Column generation algorithm

Start with a subset of columns indexed by the set $\mathcal{I} \subseteq [n]$. At each iteration $r = 1, \ldots,$

- Consider the restricted problem obtained by considering only the subset of the columns indexed by set I

$$\begin{array}{ll} \textbf{maximize} & \sum_{j \in I} \mathbf{c}_j \, \mathbf{x}_j \\ \text{subject to} & \sum_{j \in I} \mathbf{A}_j \mathbf{x}_j \leq \mathbf{b} \\ & \mathbf{x} (\in \mathbb{R}^n) \geq 0 \end{array}$$

- Solve this LP and calculate the optimal dual solution $\mathbf{p} = \mathbf{c}_B \mathbf{B}^{-1}$.
- Generate a new column $\mathbf{A}_k$ by solving the column generation subproblem $\max_{j \in [n]} \mathbf{p}\mathbf{A}_j - c_j$.

It is possible that the columns $\{\mathbf{A}_j\}$ have some underlying structures so that the above maximization over n quantities can be simplified as a simple optimization problem.

Column generation algorithm

Start with a subset of columns indexed by the set $\mathcal{I} \subseteq [n]$. At each iteration $r = 1, \ldots,$

• Consider the restricted problem obtained by considering only the subset of the columns indexed by set I

$$\begin{array}{ll} \textbf{maximize} & \sum_{j \in I} \mathbf{c}_j \, \mathbf{x}_j \\ \text{subject to} & \sum_{j \in I} \mathbf{A}_j \mathbf{x}_j \leq \mathbf{b} \\ & \mathbf{x} (\in \mathbb{R}^n) \geq 0 \end{array}$$

• Solve this LP and calculate the optimal dual solution $\mathbf{p} = \mathbf{c}_B \mathbf{B}^{-1}$.

• Generate a new column $\mathbf{A}_k$ by solving the column generation subproblem $\max_{j \in [n]} \mathbf{p}\mathbf{A}_j - c_j$.

It is possible that the columns $\{\mathbf{A}_j\}$ have some underlying structures so that the above maximization over n quantities can be simplified as a simple optimization problem.

• If the maximum > 0, add the index k of the new column to the index set $\mathcal{I}$ and continue to the next iteration. Otherwise, we obtain the optimality

Variants of column generation algorithm

- Many variants of this algorithm can be obtained by changing how the set $\mathcal{I}$ is maintained.
- One obvious variant is to simply maintain every column that has been generated so far in $\mathcal{I}$.
- Another variant is to throw away all the nonbasic columns after each iteration. There are many intermediate options.

Example: cutting stock problem

- A company supplies rolls of papers of width W. Customers requires papers of m smaller widths, e.g., b_i rolls of width w_i, for $1 \leq i \leq m$. We assume $w_i \leq W$ for every i.

- Each way to cut a roll of paper can be represented by a vector of length m, whose ith element denotes the number of rolls of papers with width w_i.

For instance, a large roll of width 70 can be cut into three rolls of $w_1 = 17$ and one roll of $w_2 = 15$. The corresponding vector is $(3, 1, 0, \ldots, 0)$.

- There are n (a finite number) ways to cut each roll of paper. Denote the jth way of cutting by a vector $\mathbf{A}_j \triangleq (a_{1,j}, a_{2,j}, \ldots, a_{m,j})$ for $j = 1, \ldots, n$.

Example: cutting stock problem

- A company supplies rolls of papers of width W. Customers requires papers of m smaller widths, e.g., b_i rolls of width w_i, for $1 \leq i \leq m$. We assume $w_i \leq W$ for every i.

- Each way to cut a roll of paper can be represented by a vector of length m, whose ith element denotes the number of rolls of papers with width w_i.

For instance, a large roll of width 70 can be cut into three rolls of $w_1 = 17$ and one roll of $w_2 = 15$. The corresponding vector is $(3, 1, 0, \ldots, 0)$.

- There are n (a finite number) ways to cut each roll of paper. Denote the jth way of cutting by a vector $\mathbf{A}_j \triangleq (a_{1,j}, a_{2,j}, \ldots, a_{m,j})$ for $j = 1, \ldots, n$.

Even if m is comparatively small, the number of feasible patterns n can be huge.

Goal: Find the minimum number of rolls of papers to be cut to satisfy all the demands of various widths.

Decision variable: Let x_j represent the rolls of papers cut in the jth way for $1 \leq j \leq n$:

Example: cutting stock problem

Model formulation:

$$
\begin{aligned}
\text{minimize} \quad & \sum_{j=1}^{n} x_j \\
\text{subject to} \quad & \sum_{j=1}^{n} x_j a_{i,j} = b_i, \quad i = 1, \ldots, m \\
& x_j \geq 0, \quad j = 1, \ldots, n
\end{aligned}
$$

We can obtain an integer solution by rounding the solution to this linear programming problem.

Example: cutting stock problem

- For $j = 1, \ldots, m$ $(m < n)$, let the jth way of cutting consist of one roll of width w_j and none of the other widths. Namely, $\mathbf{A}_j = (0, \ldots, 0, 1, 0, \ldots, 0)$ for $j = 1, \ldots, m$.
- Easy to start with a subset of variables $\{x_j\}_{j \in [m]}$ corresponding to the columns $\{\mathbf{A}_j\}_{j \in [m]}$.

Example: cutting stock problem

- For $j = 1, \ldots, m$ $(m < n)$, let the jth way of cutting consist of one roll of width w_j and none of the other widths. Namely, $\mathbf{A}_j = (0, \ldots, 0, 1, 0, \ldots, 0)$ for $j = 1, \ldots, m$.
- Easy to start with a subset of variables $\{x_j\}_{j \in [m]}$ corresponding to the columns $\{\mathbf{A}_j\}_{j \in [m]}$.
- Let $\mathbf{p} \in \mathbb{R}^m$ be the dual optimal solution to the restricted problem.
- We next seek the column with the largest positive rate of improvement: $\max\limits_{j \in [n]} \mathbf{p}^\top \mathbf{A}_j - c_j$ with $c_j = -1$.

Example: cutting stock problem

- For $j = 1, \ldots, m$ $(m < n)$, let the jth way of cutting consist of one roll of width w_j and none of the other widths. Namely, $\mathbf{A}_j = (0, \ldots, 0, 1, 0, \ldots, 0)$ for $j = 1, \ldots, m$.
- Easy to start with a subset of variables $\{x_j\}_{j \in [m]}$ corresponding to the columns $\{\mathbf{A}_j\}_{j \in [m]}$.
- Let $\mathbf{p} \in \mathbb{R}^m$ be the dual optimal solution to the restricted problem.
- We next seek the column with the largest positive rate of improvement: $\max\limits_{j \in [n]} \mathbf{p}^\top \mathbf{A}_j - c_j$

with $c_j = -1$. This is equivalent to the following column generation subproblem:

$$
\begin{aligned}
\text{maximize} \quad & \sum_{i=1}^{m} p_i z_i \\
\text{subject to} \quad & \sum_{i=1}^{m} w_i z_i \leq W \\
& z_i \geq 0, \quad \forall i = 1, \ldots, m \\
& z_i \text{integer}, \quad \forall i = 1, \ldots, m
\end{aligned}
$$

This is a Knapsack problem! Solving Knapsack problem requires some effort, but for the range of numbers that arise in the cutting stock problem, this can be done fairly efficient.

Constraint Generation Method

Constraint generation methods

For problems with large numbers of constraints, the size m of the basis is one of the biggest determinants of the speed of the simplex algorithm.

$$\boxed{\begin{array}{ll} \textbf{maximize} & Z = \mathbf{c}\,\mathbf{x} \\ \text{subject to} & \mathbf{a}_i^\top \mathbf{x} \leq \mathbf{b}_i, \quad i = 1, \ldots, m \\ & \mathbf{x}(\in \mathbb{R}^n) \geq 0 \end{array}}$$

- Recall that we only "need" the constraints that are binding at optimality.

Constraint generation methods

For problems with large numbers of constraints, the size m of the basis is one of the biggest determinants of the speed of the simplex algorithm.

$$\boxed{\begin{array}{ll} \textbf{maximize} & Z = \mathbf{c}\,\mathbf{x} \\ \text{subject to} & \mathbf{a}_i^\top \mathbf{x} \leq \mathbf{b}_i, \quad i = 1, \ldots, m \\ & \mathbf{x}(\in \mathbb{R}^n) \geq 0 \end{array}}$$

- Recall that we only "need" the constraints that are binding at optimality.
- The idea is the same as of column generation, except that now we solve the problem on a restricted row set.
- We attempt to generate an constraint from the whole set of constraints that is violated by the current "optimal" solution

Constraint generation algorithm

Start with a subset of rows indexed by the set $\mathcal{K} \subseteq [m]$. At each iteration $r = 1, \ldots,$

• Consider the restricted problem obtained by considering only the subset of the rows indexed by $\mathcal{K}$

$$\begin{array}{ll} \textbf{maximize} & \mathbf{c}\,\mathbf{x} \\ \text{subject to} & \mathbf{a}_i^\top \mathbf{x} \leq \mathbf{b}_i, \quad i \in \mathcal{K} \\ & \mathbf{x}(\in \mathbb{R}^n) \geq 0 \end{array}$$

Constraint generation algorithm

Start with a subset of rows indexed by the set $\mathcal{K} \subseteq [m]$. At each iteration $r = 1, \ldots,$

- Consider the restricted problem obtained by considering only the subset of the rows indexed by $\mathcal{K}$

$$\begin{array}{ll} \textbf{maximize} & \mathbf{c}\,\mathbf{x} \\ \text{subject to} & \mathbf{a}_i^\top \mathbf{x} \leq \mathbf{b}_i, \quad i \in \mathcal{K} \\ & \mathbf{x}(\in \mathbb{R}^n) \geq 0 \end{array}$$

- Solve this LP and calculate the optimal primal solution $\hat{\mathbf{x}}^r$.
- Generate a new row $\mathbf{a}_j$ by solving the constraint generation subproblem $\min\limits_{j \in [m]} \mathbf{b}_j - \mathbf{a}_j^\top \hat{\mathbf{x}}^r$.

Constraint generation algorithm

Start with a subset of rows indexed by the set $\mathcal{K} \subseteq [m]$. At each iteration $r = 1, \ldots,$

• Consider the restricted problem obtained by considering only the subset of the rows indexed by $\mathcal{K}$

$$\begin{array}{ll} \textbf{maximize} & \mathbf{c}\,\mathbf{x} \\ \text{subject to} & \mathbf{a}_i^\top \mathbf{x} \leq \mathbf{b}_i, \quad i \in \mathcal{K} \\ & \mathbf{x}(\in \mathbb{R}^n) \geq 0 \end{array}$$

• Solve this LP and calculate the optimal primal solution $\hat{\mathbf{x}}^r$.

• Generate a new row $\mathbf{a}_j$ by solving the constraint generation subproblem $\min_{j \in [m]} \mathbf{b}_j - \mathbf{a}_j^\top \hat{\mathbf{x}}^r$.

It is possible that the columns $\{\mathbf{a}_j\}$ have some underlying structures so that the above minimization over m quantities can be simplified as a simple optimization problem.

• If the minimum < 0, add the index of the new row to the index set $\mathcal{K}$ and continue to the next iteration. Otherwise, we obtain the optimality.

Integer programming

Integer programming (IP)

Applications of IP involves:

- the extension of LP where the divisibility assumption is dropped:
  - the units of products must be integer
  - the number of labor involvement must be integer

Integer programming (IP)

Applications of IP involves:

- the extension of LP where the divisibility assumption is dropped:
  - the units of products must be integer
  - the number of labor involvement must be integer
- a number of interrelated "yes-or-no" decisions:
  - Should we undertake a particular fixed project?
  - Should we make a particular fixed investment?
  - Should we locate a facility in a particular site?

$$x_j = \begin{cases} 1 & \text{if decision } j \text{ is yes} \\ 0 & \text{if decision } j \text{ is no} \end{cases}$$

Such variables are called binary variables or 0-1 variables.

Binary integer programming (BIP): IP problems that contain only binary variables. They are also called as **0–1 integer programming** problems.

Pure integer programming: all variables are integers

Mixed integer programming: only some variables need to be integers

A prototype example

The CALIFORNIA MANUFACTURING COMPANY is considering expansion by building a new factory in either Los Angeles or San Francisco, or perhaps even in both cities. It also is considering building at most one new warehouse, but the choice of location is restricted to a city where a new factory is being built.

Decision variables:

$x_1 \in \{0, 1\}$ represents whether to build a factory in Los Angeles,
$x_2 \in \{0, 1\}$ represents whether to build a factory in San Francisco,
$x_3 \in \{0, 1\}$ represents whether to build a warehouse in Los Angeles,
$x_4 \in \{0, 1\}$ represents whether to build a warehouse in San Francisco.

The net present value and capital required are shown below. The objective is to find the feasible combination of alternatives that maximizes the total net present value.

Yes or No question	decision variable	net present value	capital required
whether to build a factory in LA	x_1	9 million	6 million
whether to build a factory in SF	x_2	5 million	3 million
whether to build a warehouse in LA	x_3	6 million	5 million
whether to build a warehouse in SF	x_4	4 million	2 million

A prototype example

In the prototype example:

- Building at most one new warehouse: $x_3 + x_4 \leq 1$
- The choice of warehouse location is restricted to a city where a new factory is being built: $x_3 \geq x_1, x_4 \geq x_2$.

A prototype example

In the prototype example:

• Building at most one new warehouse: $x_3 + x_4 \leq 1$

• The choice of warehouse location is restricted to a city where a new factory is being built: $x_3 \geq x_1, x_4 \geq x_2$.

Two general types of "yes-or-no" decisions:

• mutually exclusive alternatives such that only one decision in the group can be yes.

• one decision is contingent on another decision such that it is allowed to be yes only if the other is yes.

Reading materials

Section 6.1-6.4, 8.1. *Introduction to Operations Research* by Hillier and Lieberman